

1. Query Processing — Overview

When you write a SQL query like:

```
SELECT name
```

```
FROM student
```

```
WHERE cgpa > 3.5;
```

The database doesn't just "run" it directly.

It must **understand**, **plan**, and **execute** it efficiently — this is **Query Processing**.

Steps in Query Processing

Step 1: Parsing and Translation

- SQL syntax is checked for correctness.
- Query is translated into **relational algebra**, a math-like language.

Example:

```
SELECT id, salary
```

```
FROM instructor
```

```
WHERE id > 50501;
```

Relational Algebra:

$\sigma_{id > 50501} (\pi_{id, salary} (instructor))$

- σ (sigma) $\rightarrow$ *Selection* (filters rows)
 - π (pi) $\rightarrow$ *Projection* (chooses columns)
-

Step 2: Optimization

Many ways exist to execute the same query.

The **optimizer** chooses the most efficient one.

Example:

1. $\sigma_{id > 50501} (\pi_{id, salary} (instructor))$
2. $\pi_{id, salary} (\sigma_{id > 50501} (instructor))$

Both give the same result but may have different **costs** depending on the storage and indexes.

 Goal $\rightarrow$ Choose the plan with **lowest cost**, using statistics like number of tuples, block size, etc.

Step 3: Evaluation

The **query execution engine** carries out the optimized plan, reads data from disk, and returns the result.

Summary

Step	Meaning	Example
Parsing	SQL $\rightarrow$ Relational algebra	SELECT $\rightarrow \sigma$ and π
Optimization	Choose cheapest way	Linear scan vs Index
Evaluation	Run the plan	Fetch data and output

2. Measures of Query Cost

DBMS needs a way to decide which plan is cheaper.

So it measures the **query cost**, mainly based on **disk I/O**.

Why Disk I/O?

Because disk access is **millions of times slower** than CPU calculations.

Important Cost Factors

Term	Meaning
Seek	Time to move disk head to correct track
Block Transfer	Time to read/write one block (usually 4–8 KB)
Block Write	Time to save result back to disk

Formula

$$\text{Cost} = (b \times t_T) + (S \times t_S)$$

Where:

- b = number of blocks read or written
 - S = number of seeks
 - t_T = time to transfer 1 block
 - t_S = time for 1 seek
-

Example

Magnetic Disk:

$t_S = 4 \text{ ms}$, $t_T = 0.1 \text{ ms}$, 100 blocks, 1 seek

$$= (100 \times 0.1) + (1 \times 4) = 14 \text{ ms} \quad (100 \times 0.1) + (1 \times 4) = 14 \text{ ms}$$

SSD (faster):

$t_S = 0.02 \text{ ms}$, $t_T = 0.002 \text{ ms} \rightarrow \text{Cost} = 0.22 \text{ ms}$

✓ So we always prefer fewer **seeks** and **block transfers**.

3. Selection Operation (Filtering Rows)

Selection = picking specific records that satisfy a **WHERE** condition.

Example:

```
SELECT * FROM student WHERE id = 1001;
```

Algorithm A1: Linear Search (Full Table Scan)

Scan **each block** and test all records.

Cost:

$$b_r + 1 \text{ seek} \quad b_r + 1 \text{ seek}$$

where b_r = number of blocks in relation r .

If record found halfway:

$$(b_r/2) + 1 \text{ seek} \quad (b_r/2) + 1 \text{ seek}$$

✓ Works for any condition, even without an index.

✗ Very slow for large tables.

Example

student has 100 blocks.

```
SELECT * FROM student WHERE id = 9001; (last record)
```

$$\text{Cost} = 1 + 100 = 101 \text{ (1 seek + 100 block reads)} \quad \text{Cost} = 1 + 100 = 101 \text{ (1 seek + 100 block reads)}$$

4. Selections Using Indices

If a table has an **index** (like a B+ Tree), we can search much faster.

Algorithm A2: Primary Index (Equality on Key)

Search for one record using index.

Cost:

$$(h_i + 1) \times (t_T + t_S) \times (h_i + 1) \times (t_T + t_S)$$

where h_i = height of index tree.

Example:

$h_i = 2$, $t_S = 5\text{ms}$, $t_T = 0.1\text{ms}$

$$\rightarrow (2+1) \times (5.1) = 15.3 \text{ ms}$$

✔ Much faster than scanning all 100 blocks.

Algorithm A3: Primary Index (Equality on Non-Key)

Multiple records (e.g., same name) on consecutive blocks.

Cost:

$$h_i(t_T + t_S) + t_S + b(t_T)h_i(t_T + t_S) + t_S + b(t_T)h_i(t_T + t_S) + t_S + b(t_T)$$

Example:

$h_i = 3$, $b = 3$, $t_S = 5\text{ms}$, $t_T = 0.1\text{ms}$

$$\rightarrow 3(5.1) + 5 + 0.3 = 20.6 \text{ ms}$$

Algorithm A5: Primary Index (Comparison)

When condition is $\geq$ or $\leq$

Example:

SELECT * FROM student WHERE id $\geq$ 25003;

Use index to find first matching record, then scan sequentially.

Algorithm A4: Secondary Index (Equality on Non-Key)

If the index is on a non-key field, each record may be on a different block.

Cost:

$$(h_i + n) \times (t_T + t_S) \times (h_i + n) \times (t_T + t_S)$$

(n = number of matching records)

✗ Can be very expensive if many matches!

✔ **Summary of Selection Methods**

Algorithm Type		Cost	Example
A1	Linear search	$b_r + 1$ seek	No index
A2	Primary index, key	$(h_i + 1)(t_T + t_S)$	id = 1001
A3	Primary index, non-key	$h_i(t_T + t_S) + t_S + b t_T$	name = 'Fatema'
A5	Primary index, comparison	find first match, then sequential scan	id $\geq$ 25003
A4	Secondary index	$(h_i + n)(t_T + t_S)$	name = 'Sajid'

 5. Sorting (External Sort-Merge)

Used when we need to **sort** data or perform operations like ORDER BY or GROUP BY.

If data doesn't fit in memory $\rightarrow$ we use **External Sorting**.

 Step 1: Run Creation

- Memory can hold **M** blocks.
- Read M blocks at a time, sort them, and write them back to disk as a **run**.

Number of runs:

$$N = \lceil br/M \rceil \quad N = \lceil b_r / M \rceil \quad N = \lceil br/M \rceil$$

Example:

$b_r = 100, M = 11 \rightarrow N = \lceil 100 / 11 \rceil = 10$ runs

Step 2: Merge Pass

- Use **M-1** blocks as input buffers and 1 block as output buffer.
- Merge runs together repeatedly.

After each pass:

New runs = $\lceil \text{Old runs} / (M-1) \rceil$

Continue until 1 final run remains (fully sorted).

Step 3: Cost Analysis

Block transfers:

$br \times (2 \times \lceil \log_{M-1}(br/M) \rceil + 1) b_r \times (2 \times \lceil \log_{M-1}(b_r / M) \rceil + 1)$

Seeks:

$2 \lceil br/M \rceil + br(2 \lceil \log_{M-1}(br/M) \rceil - 1) 2 \lceil b_r / M \rceil + b_r(2 \lceil \log_{M-1}(b_r / M) \rceil - 1) 2 \lceil br/M \rceil + br(2 \lceil \log_{M-1}(br/M) \rceil - 1)$

Example:

$b_r = 200, M = 5$

Initial runs = $\lceil 200/5 \rceil = 40$

1st merge: $40 \rightarrow 10$

2nd merge: $10 \rightarrow 3$

3rd merge: $3 \rightarrow 1$

 3 merge passes in total.

6. Join Operations

Joining = combining records from two tables based on a condition.

Example:

SELECT s.id, s.name, t.course_id

FROM student s

JOIN takes t ON s.id = t.id;

Nested Loop Join

Algorithm:

For each record in r, scan all records in s.

Cost (worst case):

$nr \times bs + br n_r \times b_s + b_r n_r \times bs + br$

Cost (best case):

If smaller relation fits in memory $\rightarrow b_r + b_s$

Example:

student = 2000 blocks, takes = 400 blocks, memory = 1 block each

$20000 \times 400 + 2000 = 8,002,000$ block transfers

$20000 \times 400 + 2000 = 8,002,000$ block transfers!

 Extremely costly!

Block Nested Loop Join

Works **block by block** instead of row by row.

Worst case:

$br \times bs + br b_r \times b_s + b_r br \times bs + br$

Best case:

$br + b_{sb_r} + b_{sbr} + bs$

- ✔ Much faster than normal nested loop join.

7. Evaluation of Expressions (Materialization & Pipelining)

So far, we've learned how to run *one* operation.

Now we learn how to run a **whole query tree** (with multiple operations).

Example Query

SELECT name

FROM instructor

JOIN department ON instructor.dept_name = department.dept_name

WHERE building = 'Watson';

Relational Algebra:

$\pi \text{ name } ((\sigma \text{ building} = \text{'Watson'}(\text{department})) \bowtie \text{instructor})$

Option 1: Materialization

- Each intermediate result is stored on disk before moving to the next step.
- Simple but slow (requires extra disk I/O).

Total cost =

(sum of all operation costs) + (cost to write intermediate results)

Example:

- Selection: 1 seek + 20 blocks
- Join: 10 seeks + 100 blocks
- Projection: 1 seek + 50 blocks
- Write costs = (1+1) seek + (10+50) blocks

Overall = (12 seeks + 170 blocks) + (2 seeks + 60 blocks) = 14 seeks + 230 blocks

Option 2: Pipelining

- Pass result directly from one operation to another **without writing to disk**.
- Much faster since it saves I/O.

Example:

Instead of saving output of selection, send rows directly to join → then to projection.

Pipelining Types

Type	Description	Also called
Demand-driven	Top operation requests next tuple from below	<i>Pull model</i>
Producer-driven	Bottom operation pushes tuples up	<i>Push model</i>

Why Not Always Pipelining?

Because some operations (like sorting or hash join) need to see **all data first** before producing output.

✔ Summary of Topic 7

Concept	Description	Example
Materialization	Store intermediate results	Easy but slow
Pipelining	Stream results directly	Fast, saves I/O

Concept	Description	Example
Pull model	Parent asks child for next tuple	Demand-driven
Push model	Child sends tuples upward	Producer-driven

Final Recap — Everything Together

Topic	Key Idea	Formula / Example
Query Processing	SQL → Relational Algebra → Optimized → Execution	3-step process
Query Cost	Disk I/O is main factor	$b \times t_T + S \times t_{Sb} \times t_T + S \times t_{Sb} \times t_T + S \times t_S$
Selection	Filtering rows	Algorithms A1–A6
Index Selection	Faster lookups using index	Depends on height (h_i)
Sorting	External sort-merge	$N = \lceil br/M \rceil$ $N = \lceil b_r/M \rceil$ $N = \lceil br/M \rceil$
Join	Combine two tables	Nested loop / block nested loop
Evaluation	Materialization vs Pipelining	Double buffering, push/pull models

Would you like me to make a **visual summary chart or diagram** (like a 1-page poster-style note) for this whole topic? It would include all formulas, key points, and algorithm types — easy to memorize before exams.

ChatGPT said:

Perfect  Meow — I see your uploaded document “**Sample Question Stor-RAID-Index-A1-A5 Algo.docx**”, and it contains **sample exam-style questions** on topics like:

- **Storage & RAID**
- **Indexing (Sparse / Dense / Primary / Secondary)**
- **Selection Algorithms (A1–A5)**
- **Query Cost Estimation**

Let’s go through **each section one by one**, explaining everything like a beginner’s guide, with diagrams (in text form), formulas, and step-by-step answers to similar examples.

Full Explanation: Storage, RAID, Indexing, and Selection Algorithms (A1–A5)

1. Storage & RAID (Redundant Array of Independent Disks)

Why RAID?

RAID combines multiple **physical disks** into one **logical storage unit** for:

- **Speed** (parallel read/write)
- **Reliability** (backup or parity)
- **Larger capacity**

RAID Levels (Basic)

RAID Level	Description	Data Protection	Example
RAID 0 (Striping)	Splits data across disks	 No protection	Fastest
RAID 1 (Mirroring)	Copies identical data to each disk	 Full copy	High reliability
RAID 5 (Striping + Parity)	Data + parity info spread across disks	 Fault tolerant	Most balanced

Example 1 (from Q1)

4 disks of 2TB each → store 9 blocks (B1–B9) using RAID 5

RAID 5 rule:

- Data is striped across all disks.

- Parity (P) is stored in a *rotating pattern* for redundancy.

Disk1 Disk2 Disk3 Disk4

B1	B2	B3	P1
B4	B5	P2	B6
B7	P3	B8	B9

💡 If any disk fails, parity helps rebuild missing data.

📺 Example 2 (RAID 1)

You need **20 TB effective storage**, each disk = 4 TB, using **RAID 1**.

RAID 1 = **mirroring**, so half of storage is for backup.

Total required = $2 \times \text{Effective Storage} = 2 \times 20 = 40$ TB
 $\text{Total required} = 2 \times \text{Effective Storage} = 2 \times 20 = 40$ TB

Each disk = 4 TB

$40 / 4 = 10$ disks total
 $40 / 4 = 10$ disks total

✅ **Answer: 10 disks needed**

📺 Example 3 (RAID 5)

Need 20 TB effective storage using RAID 5, each disk = 4 TB.

RAID 5 → 1 disk used for parity:

Effective Storage = $(N-1) \times 4$
 $\text{Effective Storage} = (N-1) \times 4$
 $(N-1) \times 4 = 20 \rightarrow N-1=5 \rightarrow N=6$
 $(N-1) \times 4 = 20 \rightarrow N-1=5 \rightarrow N=6$

✅ **6 disks required**

💬 RAID Comparison Summary

Feature	RAID 1	RAID 5
Mirroring	Yes (exact copy)	No
Parity	No	Yes
Read Speed	Fast	Fast
Write Speed	Slow (duplicate writes)	Medium
Fault Tolerance	High	High (1 disk)
Space Efficiency	50%	$(N-1)/N$

📺 2. Indexing

Indexing = like a **book index** — helps the database find data quickly without scanning the entire table.

📺 Dense vs Sparse Index

Type	Definition	Example
Dense Index	Every record has an index entry	Perfect for small tables
Sparse Index	Only some records have entries	Works with sorted data files

📺 Example (From Q3)

Customer table:

c-id Name City

C-1 Fatema Dhaka

c-id	Name	City
C-2	Sharif	Comilla
C-3	Zahid	Khulna
C-4	Abid	Dhaka
C-5	Fatema	Comilla
...	...	...

(a) Sparse index on c-id

Suppose every **third record** has an index entry.

Index Pointer

C-1	Block1
C-4	Block2
C-7	Block3
C-10	Block4

To find C-6:

- Locate the nearest smaller key (C-4)
- Scan forward until C-6 found 

(b) Index on Name

If multiple people share same name (non-key), use **secondary index**.

Name Pointers

Fatema	[C-1, C-5, C-12]
Sharif	[C-2, C-6, C-9]
Zahid	[C-3, C-7, C-10]
Abid	[C-4, C-8, C-11]

Sparse Index Rule

- Works only on **sorted files**
- Reduces index size
- Finding a record = 1 index search + sequential scan

3. Selection Algorithms (A1–A5)

These algorithms explain **how to execute WHERE conditions** efficiently.

Algorithm A1 – Linear Search (File Scan)

Check every record in every block.

Cost:

$b_r + 1 \text{ seek}_b + r + 1 \text{ seek}$

If record found halfway:

$(b_r/2) + 1 + (b_r/2) + 1 = (b_r/2) + 2$

Best when: no index exists.

Example:

100 blocks in student table

Query: SELECT * FROM student WHERE id=1001;

→ last record → 1 seek + 100 transfers = 101 I/O

■ Algorithm A2 – Primary Index (Equality on Key)

Use **index on primary key** to directly find record.

Cost:

$$(h_i+1)(t_T+t_S)(h_i+1)(t_T+t_S)$$

where h_i = index height.

Example:

Index height = 3, $t_S = 10\text{ms}$, $t_T = 0.1\text{ms}$

$$(3+1)(10.1)=40.4\text{ ms} \quad (3+1)(10.1) = 40.4 \text{ ms}$$

✓ Much faster than scanning all 100 blocks.

■ Algorithm A3 – Primary Index (Equality on Non-Key)

Multiple matches (e.g., name = 'Fatema').

Cost:

$$h_i(t_T+t_S)+t_S+b(t_T)h_i(t_T+t_S)+t_S+b(t_T)$$

where b = number of blocks with matching records.

■ Algorithm A4 – Secondary Index (Equality on Non-Key)

Each record may be in different block.

Cost:

$$(h_i+n)(t_T+t_S)(h_i+n)(t_T+t_S)$$

where n = number of matching records.

⚠ Can be slow when n is large (many duplicates).

■ Algorithm A5 – Primary Index (Comparison)

Used for queries like:

SELECT * FROM student WHERE id >= 25003;

Steps:

1. Use **index** to find first matching record ($\text{id} \geq 25003$)
2. Then **sequentially scan** from there

✓ Faster than full scan

✗ Only works if table sorted on that attribute.

🧩 4. Sample Cost Calculation (from your Q8)

Student relation = 1000 blocks

Seek time = 10 ms, block transfer = 0.1 ms

Query:

SELECT * FROM student WHERE id > 1801111042

Student sorted on id

500 blocks satisfy condition

Index height = 3

Step 1: Find first matching record (via index)

$$\text{Index cost} = h_i \times (t_S + t_T) = 3 \times (10 + 0.1) = 30.3 \text{ ms}$$

Step 2: Read remaining 500 blocks sequentially

Sequential scan cost = $500 \times 0.1 = 50 \text{ ms}$ \text{Sequential scan cost} = 500 \times 0.1 = 50 \text{ ms}

Step 3: Add initial seek

$+1 \times 10 = 10 \text{ ms} + 1 \times 10 = 10 \text{ ms} + 1 \times 10 = 10 \text{ ms}$

✓ **Total = 30.3 + 10 + 50 = 90.3 ms**

5. RAID, Index, and Query Summary

Topic	Key Formula / Rule	Example
RAID 1	Total = 2 × Effective	20TB → 10 disks
RAID 5	Effective = (N−1)×size	N=6 for 20TB
Sparse Index	1 entry per few blocks	Find nearest smaller key
Dense Index	Every record	All entries indexed
A1	Linear search	$b_r + 1$
A2	Primary index on key	$(h_i + 1)(tT + tS)$
A3	Primary index non-key	$h_i(tT + tS) + tS + b(tT)$
A4	Secondary index	$(h_i + n)(tT + tS)$
A5	Primary index comparison	find first, scan sequentially

✓ You Now Understand:

- RAID Levels (0, 1, 5) and how to calculate disk needs
- Dense vs Sparse Index and lookup process
- All Selection Algorithms (A1–A5) with cost math
- Query cost estimation using seek and transfer times

When to use **A1, A2, or A5**

- ✓ How to **calculate cost** (step-by-step)
- ✓ How to **recognize** which algorithm fits a query
- ✓ 20 solved examples with explanations

🧠 QUICK CONCEPTS RECAP (BEFORE EXAMPLES)

⚙️ Algorithm A1 – Linear Search (Full Table Scan)

When to use:

- No index available on the attribute in WHERE clause
- OR the data file is *unsorted*
- OR you are checking *many records*

Process:

Scan every block, test every record.

Cost Formula:

Cost = $b_r + 1$ seek \text{Cost} = b_r + 1 \text{ seek}

If searching for a single key:

Avg cost = $(b_r/2) + 1$ \text{Avg cost} = (b_r/2) + 1

⚙️ Algorithm A2 – Primary Index, Equality on Key

When to use:

- There is an **index** on the column (e.g., id)
- Condition is **equality (=)**

- Column is **unique** (primary key)

Process:

Use index → go directly to record.

Cost Formula:

$$\text{Cost} = (h_i + 1)(t_S + t_T)$$

Where

h_i = index height

t_S = seek time

t_T = block transfer time

 Algorithm A5 – Primary Index, Comparison

When to use:

- There's an **index on the attribute**
- The table is **physically sorted** by that attribute
- Condition is **range-based** ($\geq$, $\leq$, $>$, $<$)

Process:

Use index to locate first matching record → then **sequentially scan** the rest.

Cost Formula:

$$\text{Cost} = (h_i + 1)(t_S + t_T) + b_s(t_T)$$

Where b_s = number of blocks with matching records

 LET'S PRACTICE — 20 EXAMPLES (Step-by-Step)

☐ **Common Data Setup (used for most examples)**

Unless otherwise mentioned:

Parameter	Symbol	Value
Relation size	b_r	100 blocks
Seek time	t_S	10 ms
Block transfer	t_T	0.1 ms
Index height	h_i	3
Table sorted on id	☒	Yes

 A1 Examples – Linear Search



Query: SELECT * FROM student WHERE name = 'Fatema';

(No index on name)

→ Use **A1**

$$\text{Cost} = b_r + 1 = 100 + 1 = 101 \text{ I/Os}$$

Time:

$$101 \times 0.1 + 1 \times 10 = 20.1 \text{ ms}$$



Query: SELECT * FROM student WHERE cgpa > 3.5;

No index → **A1**

Same formula:

$$= 100 + 1 = 101 \text{ I/Os} \rightarrow 20.1 \text{ ms}$$

3

Query: SELECT * FROM student WHERE id = 5001;

No index → **A1**

Avg case:

$$(100/2)+1=51 \text{ I/Os} \\ (100/2) + 1 = 51 \text{ I/Os} \\ (100/2)+1=51 \text{ I/Os} \\ = (51 \times 0.1 + 10) = 15.1 \text{ ms}$$

4

Query: SELECT * FROM student WHERE dept = 'CSE' AND year = 2024;

No index → must scan all → **A1**

Cost = 101 I/Os

5

Query: SELECT * FROM student; (no WHERE)

→ Must scan all blocks.

A1

Cost = 100 + 1 = 101 I/Os

✓ **How to recognize A1:**

- No index available
- Or condition is not on indexed column
- Or you need *most records* (then index gives no benefit)

● **A2 Examples – Primary Index, Equality on Key**

6

Query: SELECT * FROM student WHERE id = 1801101;

Index on id (primary key).

Use A2

$$\text{Cost} = (h_i + 1)(t_S + t_T) = (3 + 1)(10 + 0.1) = 40.4 \text{ ms} \\ \text{Cost} = (h_i + 1)(t_S + t_T) = (3 + 1)(10 + 0.1) = 40.4 \text{ ms} \\ \text{Cost} = (h_i + 1)(t_S + t_T) = (3 + 1)(10 + 0.1) = 40.4 \text{ ms}$$

7

Query: SELECT name FROM student WHERE id = 5;

Same as above → **A2**

= 40.4 ms

8

Query: SELECT cgpa FROM student WHERE id = 1801122;

Primary key → **A2**

Same formula → 40.4 ms

9

Query: SELECT * FROM instructor WHERE instructor_id = 3005;

Assume index height = 2

$$(2+1)(10.1)=30.3 \text{ ms} \\ (2+1)(10.1) = 30.3 \text{ ms} \\ (2+1)(10.1)=30.3 \text{ ms}$$

10

Query: SELECT * FROM employee WHERE emp_no = 25;

Index height 4

$$(4+1)(10.1)=50.5 \text{ ms} \quad (4+1)(10.1) = 50.5 \text{ ms} \quad (4+1)(10.1)=50.5 \text{ ms}$$

✓ How to recognize A2:

- There is a **primary index** on the attribute.
 - Condition uses = (equality).
 - Attribute is **unique** (primary key).
-

● A5 Examples – Primary Index, Comparison

11

Query: SELECT * FROM student WHERE id >= 25003;

Table sorted on id → **A5**

Blocks satisfying condition: 40

$$\begin{aligned} \text{Cost} &= (h_i + 1)(t_S + t_T) + b_s \times t_T \\ \text{Cost} &= (3+1)(10.1) + 40(0.1) = 40.4 + 4 = 44.4 \text{ ms} \\ &= (3+1)(10.1) + 40(0.1) = 40.4 + 4 = 44.4 \text{ ms} \end{aligned}$$

12

Query: SELECT * FROM student WHERE id <= 3000;

50 blocks satisfy condition.

Same formula → $40.4 + 5 = 45.4 \text{ ms}$

13

Query: SELECT * FROM student WHERE id > 1001;

70 blocks match.

$$40.4 + 7 = 47.4 \text{ ms} \quad 40.4 + 7 = 47.4 \text{ ms}$$

14

Query: SELECT * FROM student WHERE id < 9000;

95 blocks match.

$$40.4 + 9.5 = 49.9 \text{ ms} \quad 40.4 + 9.5 = 49.9 \text{ ms}$$

15

Query: SELECT * FROM student WHERE id >= 50501;

Table sorted on id → 20 blocks match.

$$(3+1)(10.1) + 2 = 40.4 + 2 = 42.4 \text{ ms} \quad (3+1)(10.1) + 2 = 40.4 + 2 = 42.4 \text{ ms}$$

✓ How to recognize A5:

- Condition is **range-based** (>, <, >=, <=)
 - Attribute is **primary key**
 - Table is **physically sorted** by that key
 - Index exists on that key
-

● Mixed Examples (Compare which algo fits)

16

Query: SELECT * FROM student WHERE city = 'Dhaka';

- If **no index on city** → A1
- If **secondary index on city** → not A2/A5 (it's A4)

→ Here assume **no index**, so A1 → 101 I/Os

17

Query: SELECT * FROM student WHERE id = 3001;

→ id = primary key → **A2**

Cost = (3+1)(10.1) = 40.4 ms

18

Query: SELECT * FROM student WHERE id > 5000;

→ id indexed, sorted → **A5**

Suppose 30 blocks match:

$(3+1)(10.1)+30(0.1)=40.4+3=43.4$ ms
 $(3+1)(10.1) + 30(0.1) = 40.4 + 3 = 43.4$ ms

19

Query: SELECT * FROM student WHERE name = 'Sajid';

→ No index on name → **A1**

→ 101 I/Os → 20.1 ms

20

Query: SELECT * FROM student WHERE id >= 25009;

→ Table sorted, index on id → **A5**

Suppose 10 blocks match:

$(3+1)(10.1)+1=40.4+1=41.4$ ms
 $(3+1)(10.1) + 1 = 40.4 + 1 = 41.4$ ms

SUMMARY TABLE (Quick Reference)

#	Query	Index Present	Condition	Algorithm	Cost Formula	Example Result
1	name='Fatema'	✗	=	A1	b+1	101 I/O
2	cgpa>3.5	✗	>	A1	b+1	101
3	id=5001	✗	=	A1	b/2+1	51
6	id=1801101	✓	=	A2	(h+1)(tS+tT)	40.4 ms
9	emp_no=25	✓	=	A2	(4+1)(tS+tT)	50.5 ms
11	id>=25003	✓	>=	A5	(h+1)(tS+tT)+b_s*tT	44.4 ms
13	id>1001	✓	>	A5	same	47.4 ms
15	id>=50501	✓	>=	A5	same	42.4 ms
16	city='Dhaka'	✗	=	A1	b+1	101
19	name='Sajid'	✗	=	A1	b+1	101

HOW TO DECIDE WHICH ALGORITHM TO USE

Condition Type	Index Exists?	Sorted on Attribute?	Algorithm
Equality (=)	No	Doesn't matter	A1

Condition Type	Index Exists?	Sorted on Attribute?	Algorithm
Equality (=)	Yes, Primary Index	Yes	A2
Comparison (>, <, >=, <=)	Yes, Primary Index	Yes	A5
Comparison	No index	Doesn't matter	A1
Equality on Non-Key	Secondary Index	No	A4 (not in your set)

✓ Simple Trick to Remember

Query	Shortcut	Algo
No index → Scan	"Search all"	A1
= on key	"Direct hit"	A2
>= or <=	"Range scan"	A5

WHY MULTIPLE ALGORITHMS?

Because databases can **choose different physical access methods** depending on:

- Whether the table has an **index**
- Whether the attribute is **sorted**
- Whether the condition matches an **index type**
- How many records the condition will match (**selectivity**)

So, *the same query* could be executed by:

- Scanning the **entire table (A1)**
- Using an **index lookup (A2 or A5)**

The **optimizer** estimates the cost of both and picks the cheaper.

⚙ Example 1 — id > 50501

```
SELECT id, salary
FROM instructor
WHERE id > 50501;
```

◆ Option 1 — A1 (Linear Search)

Scan all records and test each id.

Cost:

$br + 1b_r + 1br + 1$

If table has 1000 blocks → **1001 I/Os**

◆ Option 2 — A5 (Primary Index, Comparison)

Table is sorted on id, index exists.

Steps:

1. Use index to find first record id >= 50501
2. Sequentially scan the rest of the file

If 1000 blocks, and 200 blocks have id > 50501

→ **Cost = $(h_i + 1)(t_S + t_T) + 200 \times t_T$**

◆ Which is better?

Let's calculate:

- $h_i=3, t_S=10, t_T=0.1$ $h_i = 3, t_S = 10, t_T = 0.1$

A1 cost:

$1001 \times 0.1 + 10 = 110.1 \text{ ms}$ $1001 \times 0.1 + 10 = 110.1 \text{ ms}$

A5 cost:

$(3+1)(10.1)+200(0.1)=40.4+20=60.4$ ms
 $(3+1)(10.1) + 200(0.1) = 40.4 + 20 = 60.4$ \text{ ms}

✅ A5 is cheaper → DBMS will choose A5.

💡 **Rule of Thumb:**

- If **few records** match condition → index (A5) is better.
- If **most records** match condition → full scan (A1) is better.

⚙️ **Example 2 — id = 2001**

SELECT * FROM student WHERE id = 2001;

♦ **Option 1 — A1**

Full table scan.

If 100 blocks, average 50 reads + 1 seek = 51 I/Os.

♦ **Option 2 — A2**

Primary index (height = 3)

$(3+1)(10.1)=40.4$ ms
 $(3+1)(10.1) = 40.4$ \text{ ms}

♦ **Compare**

- $A1 = (50 \times 0.1) + 10 = 15$ ms
- $A2 = 40.4$ ms

😬 Here, *A1 is cheaper!* (since table is small and seek time dominates)

✅ So, if table is **small**, linear search (A1) can outperform index (A2)!

💬 **Rule:**

- **Small tables** → scanning is faster (A1)
- **Large tables** → indexing is faster (A2)

⚙️ **Example 3 — id < 5001**

SELECT * FROM student WHERE id < 5001;

- Table sorted on id
- Index available

Possible algorithms:

- **A1:** Linear search
- **A5:** Primary index comparison

If 90% of table satisfies (id < 5001),

then sequential scanning from start is nearly the same as full scan → $A1 \approx A5$.

✅ Optimizer may still prefer **A1** if too many records match.

⚙️ **Example 4 — Mixed Attributes**

SELECT * FROM student

WHERE id > 2001 AND name = 'Fatema';

- id → indexed, sorted (A5 possible)
- name → not indexed (A1)

Possible:

1. **Use A5** on id to get 1000 candidates, then filter for name
2. **A1** → scan all, check both conditions

Which is better?

→ Depends on **how selective** id > 2001 is.

If it returns 90% of table → scanning all (A1) is cheaper.

If it returns 1% → A5 is cheaper.

🌟 General Rule — When Queries Can Use Multiple Algorithms

Query Type	Available Algorithms	DBMS Chooses Based On
WHERE id = constant	A1, A2	Table size vs index height
WHERE id > constant	A1, A5	Fraction of records that match
WHERE name = 'x' (no index)	Only A1	—
WHERE name = 'x' (with secondary index)	A1, A4	Cost of A4
WHERE key1 = X AND key2 > Y	A1, A2, A5	Which filter is selective

🎨 Let's See Cost Comparison Example

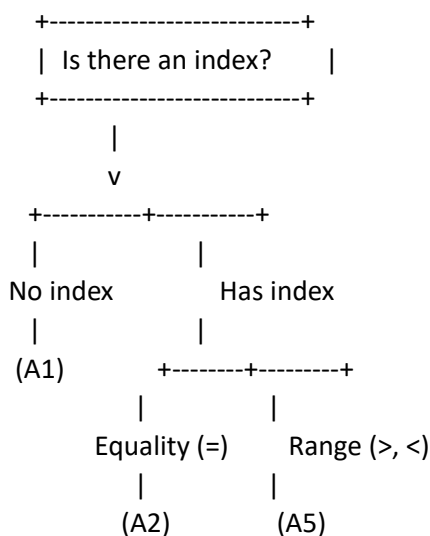
Case Table Size (blocks) Matching % Algorithm Estimated Cost (ms) Faster

1	100	1% match	A2	40.4	✓ A2
2	100	90% match	A1	20.1	✓ A1
3	1000	1% match	A5	60	✓ A5
4	1000	90% match	A1	110	✓ A1
5	20	1% match	A1	12	✓ A1

💡 Observation:

- Small table → A1
- Large table + few matches → A2/A5

🔍 Quick Decision Flow (Super Simple)



Then check **selectivity**:

- If result set is large → A1
- If result set is small → A2/A5

⚡ How to Answer in Exams

If a question says:

“The table is sorted on id. Which algorithm(s) can be used for this query and what’s the cost?”

Write like this 🗨️

Example Answer Format

Query: SELECT * FROM student WHERE id > 5001

✅ Possible Algorithms:

- **A1 (Linear Scan)** — because any query can always use scanning.
- **A5 (Primary Index, Comparison)** — because index exists and table is sorted.

✅ Cost Calculation:

- For A1: $b_r + 1 = 100 + 1 = 101$ I/Os
- For A5: $(h_i + 1)(tS + tT) + b_s \times tT = 40.4 + 20 = 60.4$ ms

✅ Best Choice: A5 (lower cost)

🌸 Summary Table (When Both Possible)

Query	Possible Algorithms	DB Chooses	Why
id = constant	A1, A2	A2 (if table large)	equality key
id > constant	A1, A5	A5 (if few matches)	range, sorted
id < constant	A1, A5	A1 (if many matches)	scanning cheaper
name = 'x'	A1, A4	A4 if indexed	secondary index
no WHERE	A1 only	A1	must scan all

🧠 RAID BASICS — THE BIG IDEA

RAID means: combining multiple physical disks into one logical storage system for:

- 🌀 **Speed** (parallel access)
- 💾 **Reliability** (fault tolerance)
- 📀 **Capacity**

The disks work **together**, and depending on RAID level, data may be **split (striped)**, **duplicated (mirrored)**, or **stored with parity bits** for recovery.

⚙️ RAID COMPONENTS YOU MUST KNOW

Term	Meaning
Block (or stripe)	Small chunk of data (e.g., 4 KB or 64 KB) stored per disk
Striping	Splitting data across multiple disks to increase speed
Mirroring	Duplicating data on another disk for fault tolerance
Parity	Extra data calculated (like a checksum) that helps reconstruct lost data when a disk fails

⚡ RAID LEVELS SUMMARY TABLE

Level	Technique	Fault Tolerance	Usable Capacity	Speed	Write Performance
RAID 0	Striping only	❌ None	100% (all disks)	🚀 Fast	🚀 Fast
RAID 1	Mirroring	✅ 1 disk (per pair)	50%	✅ Good	⚠️ Slower (double write)
RAID 5	Striping + Parity	✅ 1 disk	(N-1)/N	🚀 Fast read	⚠️ Medium (parity write)
RAID 6	Double Parity	✅ 2 disks	(N-2)/N	🚀 Read	⚠️ Slower writes

🔍 RAID 0 — Striping (No Parity, No Mirror)

🌸 Idea

Data is split (striped) across all disks.

Disk 1 Disk 2 Disk 3 Disk 4

B1 B2 B3 B4

B5 B6 B7 B8

✅ Advantages

- Very fast (reads/writes in parallel)
- 100% usable storage

❌ Disadvantage

- No fault tolerance — if one disk fails, all data is lost.

📊 Example

4 disks, each 2 TB

→ **Total usable** = $4 \times 2 \text{ TB} = 8 \text{ TB}$

💾 RAID 1 — Mirroring

🌱 Idea

Data is duplicated (mirrored) on another disk.

Disk 1 Disk 2

B1 B1

B2 B2

✅ Advantage

- Excellent reliability — if one disk fails, the mirror still has the data.

❌ Disadvantage

- Only 50% usable storage.

📊 Example

10 disks, 2 TB each

→ Every 2 disks are a mirror pair → usable = $10/2 \times 2 = 10 \text{ TB}$

📊 RAID 1 Exam Formula

Usable Capacity = $N/2 \times \text{disk size}$
Disks needed = $2 \times \frac{\text{desired capacity}}{\text{disk size}}$

🌱 RAID 5 — Striping with Distributed Parity

This is the most exam-tested level.

💡 Idea

Data is striped across all disks, and **parity** (P) is stored in a *rotating pattern* across disks.

Parity = XOR of data bits in that stripe.

📊 Parity Bit Concept (XOR)

Let's say a stripe has 3 data blocks (D1, D2, D3).

Parity (P) = bitwise XOR of all data blocks.

D1 D2 D3 Parity P

1 0 1 $(1 \oplus 0 \oplus 1) = 0$

0 1 1 $(0 \oplus 1 \oplus 1) = 0$

→ If any one disk fails, missing data = XOR of remaining disks.

Example Recovery:

If D2 is lost $\rightarrow D2 = D1 \oplus D3 \oplus P$

🌸 RAID 5 Layout Example (4 Disks, B1–B9)

Disk1 Disk2 Disk3 Disk4

B1 B2 B3 P1

B4 B5 P2 B6

B7 P3 B8 B9

Each stripe has 3 data blocks + 1 parity block.

Parity rotates $\rightarrow$ each disk stores parity for one stripe.

📊 RAID 5 Formulas

Usable Capacity = $(N-1) \times \text{disk size}$ $\text{Usable Capacity} = (N-1) \times \text{disk size}$

Storage Efficiency = $\frac{N-1}{N}$ $\text{Storage Efficiency} = \frac{N-1}{N}$

Fault Tolerance = 1 disk failure $\text{Fault Tolerance} = 1 \text{ disk failure}$

📊 Example 1

5 disks of 2 TB each in RAID 5.

Find usable capacity and total capacity loss due to parity.

Usable = $(5-1) \times 2 = 8 \text{ TB}$ $\text{Usable} = (5-1) \times 2 = 8 \text{ TB}$

Total = 10 TB, so parity uses 2 TB.

✅ Answer: 8 TB usable, 2 TB parity overhead.

📊 Example 2

Need 20 TB effective storage using 4 TB disks.

$(N-1) \times 4 = 20 \Rightarrow N-1 = 5 \Rightarrow N = 6$ $(N-1) \times 4 = 20 \Rightarrow N-1 = 5 \Rightarrow N = 6$

✅ Need 6 disks.

📊 Example 3 (How to rebuild with parity)

Stripe 1:

D1=1011, D2=1101, D3=0110 $\rightarrow$ parity = XOR of all.

Compute parity:

D1: 1011

D2: 1101

D3: 0110

XOR = 1000

P = 1000

If D2 is lost, recover it:

$D2 = D1 \oplus D3 \oplus P$

$= 1011 \oplus 0110 \oplus 1000 = 1101$ ✅

📊 RAID 6 — Striping with Double Parity

Like RAID 5 but stores **two parity blocks** per stripe (P & Q).

So even if two disks fail, data can be rebuilt.

📊 RAID 6 Formula

Usable Capacity = $(N-2) \times \text{disk size}$ \text{Usable Capacity} = (N - 2) \times \text{disk size}

Fault Tolerance = 2 disks failure \text{Fault Tolerance} = 2 \text{ disks failure}

Example

6 disks of 2 TB each, RAID 6.

Usable = $(6-2) \times 2 = 8$ TB \text{Usable} = (6 - 2) \times 2 = 8 \text{ TB}

Efficiency = $8/12 = 66.6\%$

✓ 2 disks used for parity.

RAID 10 (RAID 1 + 0)

Mirror each pair (RAID 1), then stripe across pairs (RAID 0).

Pair Disks Mirrored?

1 D1,D2 Yes

2 D3,D4 Yes

3 D5,D6 Yes

Data striped across pairs:

Stripe1: B1 (Pair1)

Stripe2: B2 (Pair2)

Stripe3: B3 (Pair3)

✓ Fast + Reliable

✗ 50% usable (like RAID 1)

EXAM-TYPE PROBLEMS & HOW TO SOLVE

Q1:

You have 4 disks (1 TB each) in RAID 0. Find total capacity and fault tolerance.

Step 1: RAID 0 → no parity, no mirror.

Step 2: Capacity = all disks combined.

$= 4 \times 1 = 4$ TB = $4 \times 1 = 4$ \text{TB} = 4 \times 1 = 4 \text{ TB}

Fault tolerance = 0 (if 1 disk fails → all data lost). ✓

Q2:

You have 6 disks (2 TB each) in RAID 1.

Find usable capacity and fault tolerance.

RAID 1 → mirrored pairs.

Usable = $(6/2) \times 2 = 6$ TB \text{Usable} = (6/2) \times 2 = 6 \text{ TB}

Fault tolerance = 1 disk per pair (so 3 disks could fail, 1 per pair). ✓

Q3:

6 disks of 4 TB each, RAID 5.

(a) How much storage is usable?

(b) How many disks' space is used for parity?

(a)

$(6-1) \times 4 = 20$ TB $(6-1) \times 4 = 20$ \text{TB}

(b)

1 disk used for parity → 4 TB parity overhead.

✓ Usable 20 TB, 4 TB parity.



Q4:

5 disks, 3 TB each, RAID 6. Find usable capacity.

RAID 6 uses 2 disks for parity.

$$(5-2) \times 3 = 9 \text{ TB}$$

✓ Usable = 9 TB.



Q5:

Need 16 TB usable capacity, using 2 TB disks in RAID 5.

How many disks needed?

$$(N-1) \times 2 = 16 \Rightarrow N-1=8 \Rightarrow N=9$$

✓ Need 9 disks.



Q6:

Which RAID gives best read speed and reliability together?

✓ **RAID 10** — because:

- Striping (speed)
- Mirroring (fault tolerance)
- Rebuild is faster than RAID 5/6



HOW TO WRITE RAID ANSWERS IN EXAM

Always include:

1. **Identify** RAID level
2. **Write formula** for usable capacity
3. **Substitute values** (show step)
4. **Mention fault tolerance**
5. **(If RAID-5 or 6)** show parity rotation or XOR concept briefly



Example: How to Write Full Exam Answer

Q: 5 disks, 2 TB each, RAID 5. Calculate total, usable, and parity storage.

Show how parity is distributed.

Answer:

RAID 5 uses striping with distributed parity.

Each stripe has (N-1) data blocks + 1 parity block (rotated among disks).

Disk1 Disk2 Disk3 Disk4 Disk5

B1 B2 B3 B4 P1

B5 B6 B7 P2 B8

B9 B10 P3 B11 B12

...

Total capacity = $5 \times 2 = 10$ TB

Usable capacity = $(5-1) \times 2 = 8$ TB

Parity overhead = 2 TB (one disk worth)

Fault tolerance = 1 disk

✓ Answer complete.



How Parity Calculation Appears in Theory Questions

Bit-level XOR parity:

D1 D2 D3 Parity (P)

1 0 1 0
0 1 1 0
1 1 0 0
0 0 1 1

If any one disk fails → missing block = XOR of all others.

QUICK SUMMARY (CHEAT-SHEET)

RAID Technique	Usable	Parity	Fault Tolerance	Example Formula
0 Striping	$N \times \text{size}$	None	0	—
1 Mirroring	$(N/2) \times \text{size}$	Copy	1 per pair	$\text{Disks} = 2 \times (\text{Req}/\text{size})$
5 Striping + 1 parity	$(N-1) \times \text{size}$	1 disk	1 disk	$N = (\text{Req}/\text{size}) + 1$
6 Striping + 2 parity	$(N-2) \times \text{size}$	2 disks	2 disks	$N = (\text{Req}/\text{size}) + 2$
10 Mirroring + Striping	$(N/2) \times \text{size}$	Mirror	1 per pair	Same as RAID 1

◆ First — What is a Secondary Index?

A **secondary index** is an **extra index** created on a **non-primary key attribute** (i.e., not the main sorting/searching key of the data file).

It provides an **alternate way** to access data besides the primary key.

Example:

Suppose a student file is stored (clustered) on **Roll Number (Primary key)**, but you also want to search by **Name** — then you create a **secondary index** on *Name*.

◆ Dense vs Sparse Index

Type	Definition	Example
Dense Index	Has one index entry for every record in the data file.	If 1,000 records → 1,000 index entries.
Sparse Index	Has index entries only for some records , usually the first record of each block .	If 1,000 records stored in 100 blocks → ~100 index entries.

◆ Why Secondary Index must be Dense

Let's understand **why sparse indexing doesn't work** here:

1. **Data file is not sorted on the secondary key**
 - The data file is physically ordered on the **primary key**, not on the **secondary key**.
 - So, the values of the secondary key are scattered randomly across the file.
2. **Sparse index needs sorted order**
 - Sparse indexes rely on the **data file being sorted** on the same key.
 - Without sorted order, we can't predict where a key's block starts → we can't skip blocks safely.
3. **Need to locate exact record(s)**
 - When you search using a secondary key, you need to find **every record** having that key (maybe multiple records).
 - To do that, you must have a **pointer for each record** → hence, **dense index**.

◆ Summary Table

Property	Primary Index	Secondary Index
Based on	Primary key (sorting key)	Non-primary attribute

Property	Primary Index	Secondary Index
File order	Ordered on key	Not ordered
Type of index	Can be sparse or dense	Must be dense
Reason	Sorted file helps locate blocks	Unsorted file → must point to every record

Example

Roll No (Primary Key) Name (Secondary Key)

101	Alice
104	Bob
102	Alice
105	Charlie

- File is **stored in order of Roll No**, not Name.
 - If you search for “Alice”, her records (101 and 102) are scattered.
 - So index on Name must point to **both records individually** → **dense index**.
-

Final Answer:

A secondary index must be **dense** because the data file is **not ordered** on the secondary key.

Therefore, to locate any record efficiently, the index must contain **an entry for every record** with its corresponding pointer.